UNIVERSIDAD EAFIT · ST1630 SISTEMAS INTENSIVOS EN DATOS · PROYECTO FINAL

CENTINELA

Detección de fraude en tiempo real y monitoreo SARLAFT para una billetera digital colombiana
Entregable 1: diagnóstico, requisitos y arquitectura

Nombre del proyecto	Centinela
Dominio	Fintech colombiana: pagos inmediatos (Bre-B, P2P, QR) y detección de fraude transaccional en **Chiva Pay**, una SEDPE ficticia con escala comparable a las billeteras digitales del país
Profesor	Andrés Sacre Alzate · csanch35@eafit.edu.co
Fecha de entrega	30/09/2026 · antes del inicio de S13
Repositorio	https://github.com/Sebaxdenc/centinela

Integrante

#	Nombre completo	Código EAFIT	Responsabilidad
1	Sebastian Andres Medina Cabezas	1000317163	Todo el sistema (ver Sección 5)

Chiva Pay es una empresa ficticia: sus volúmenes son supuestos de diseño, calibrados con cifras públicas del sector (Anexo D). En el Entregable 2 los datos los produce un simulador que inyecta patrones de fraude etiquetados.

1. Definición del problema

1.1 Contexto de negocio

Chiva Pay es una SEDPE (Sociedad Especializada en Depósitos y Pagos Electrónicos) con 9 millones de clientes, de los cuales 5,5 millones usan la app cada mes. Ofrece depósito electrónico, transferencias P2P, transferencias inmediatas por **Bre-B** con llaves, pagos QR en comercios, recargas y retiros en corresponsales. Mueve en promedio 9 millones de transacciones al día.

Desde que Bre-B entró en operación (septiembre de 2025), las transferencias entre entidades llegan en segundos y a cualquier hora. Eso cambió la forma de robar: el delincuente toma el control de una cuenta (SIM swapping, phishing o vishing), registra un dispositivo nuevo y la vacía con una ráfaga de transferencias a **cuentas mula**, que a su vez reenvían el dinero en minutos. En Colombia se registraron más de 218.000 reclamaciones por fraude en canales financieros solo en el primer semestre de 2025. Además, cada peso defraudado le cuesta a la entidad 3,68 veces su valor una vez se suman investigación, operación, regulación y experiencia del cliente (Anexo D).

Hoy el equipo de fraude de Chiva Pay trabaja con un reporte T+1 que se genera de noche sobre una réplica de PostgreSQL. Un ataque se detecta entre 14 y 30 horas después de ocurrir, cuando el dinero ya salió del ecosistema. El Oficial de Cumplimiento arma en hojas de cálculo los reportes de operaciones inusuales que exige el SARLAFT, y nadie puede decir con datos qué reglas de detección sirven y cuáles solo generan ruido.

> **Problema.** Los analistas de fraude de Chiva Pay necesitan recibir una alerta priorizada **en menos de 10 segundos (p95)** desde que ocurre una transacción sospechosa, para retener la cuenta antes de la segunda transferencia del ataque. Además, el área de riesgo necesita medir a diario, con datos de máximo 30 minutos de antigüedad, qué reglas detectan fraude real, cuánto dinero se evitó perder y qué clientes deben reportarse a la UIAF.

Metas de negocio (línea base → objetivo): tiempo de detección de ~20 h → < 10 s p95 · porcentaje del monto atacado que se retiene antes de salir: sin medición → ≥ 60 % · precisión de las alertas de severidad ALTA: desconocida → ≥ 30 %, con máximo 1.500 alertas ALTA al día para 25 analistas en turnos 24/7 · preparación del reporte SARLAFT: 2 días-persona por mes → reporte diario automático.

1.2 Usuarios y decisiones

Usuario	Decisión que toma con los datos	Cómo consume	Preguntas
Analista de fraude (25, turnos 24/7)	Bloquear o liberar una cuenta retenida, llamar al cliente, marcar la alerta como fraude o falso positivo	Consola (Streamlit + FastAPI sobre Cassandra)	Q1, Q2, Q3
Motor de bloqueo del core (sistema)	Retener de forma preventiva y automática la cuenta cuando la severidad es ALTA	Consumer group de `fraude.alertas.v1`	Q1
Líder de riesgo de fraude	Ajustar umbrales y activar o desactivar reglas, dimensionar turnos	Superset / SQL en Trino sobre Gold	Q4, Q5, Q6
Oficial de Cumplimiento	Decidir qué operaciones se reportan como ROS (Reporte de Operación Sospechosa) a la UIAF	Reporte SARLAFT diario (desde Gold)	Q7
Gerencia de producto	Definir límites por canal, exigir autenticación reforzada en comercios o municipios de riesgo	Superset	Q5, Q8

1.3 Preguntas que el sistema debe responder

ID	Pregunta (específica y medible)	Tipo	Frescura	Dónde se responde
Q1	¿Qué transacciones de los últimos segundos tienen score ≥ 0,8, de qué cliente son y qué regla las disparó?	Operacional	≤ 10 s p95	[P2] → `fraude.alertas.v1`, `cola_alertas`
Q2	Para un cliente con alerta: ¿cuántas transacciones, qué monto y cuántas contrapartes distintas acumula en los últimos 10 min y 1 h, comparado con su promedio de 90 días?	Operacional	≤ 10 s	`alertas_por_cliente` + `perfil_riesgo`
Q3	¿Qué cuentas recibieron dinero de ≥ 8 remitentes distintos en 1 h y reenviaron ≥ 80 % de lo recibido en menos de 30 min (patrón de cuenta mula)?	Operacional	≤ 10 s	[P2] regla R3

ID	Pregunta (específica y medible)	Tipo	Frescura	Dónde se responde
Q4	¿Cuál es la precisión (alertas confirmadas / alertas totales) y el monto evitado de cada regla R1–R6, por semana y por versión de umbral?	Analítica	diaria	Gold: `fact_alerta` ⋈ `dim_regla`
Q5	¿Cuál es la tasa de fraude confirmado (número y monto) por canal, franja horaria y departamento en los últimos 90 días?	Analítica	≤ 30 min	Gold: `fact_transaccion`
Q6	¿Cuáles son el p50 y el p95 del tiempo evento → alerta y alerta → decisión del analista, por turno?	Analítica	≤ 30 min	Gold: `fact_alerta`
Q7	¿Qué clientes acumulan en el mes operaciones inusuales según SARLAFT: fraccionamiento, contrapartes en listas restrictivas o montos por encima del P99 de su perfil?	Cumplimiento	diaria, 06:00	Gold → reporte ROS
Q8	¿Qué comercios (MCC) y municipios concentran el mayor monto defraudado por pago QR en los últimos 30 días?	Analítica	≤ 30 min	Gold: `fact_transaccion` ⋈ `dim_comercio`

1.4 ¿Por qué una arquitectura distribuida y no una base relacional?

Dimensión	Magnitud en Chiva Pay (supuestos de diseño)	Consecuencia para la arquitectura
Volumen	9 M transacciones y 86 M eventos de app al día. En picos (quincena, pago de prima, días sin IVA) hay 1.800 tx/s y 18.000 ev/s, es decir, ≈ 20.000 eventos/s.	Un solo nodo no absorbe esa tasa de escritura mientras hace ventanas con estado. Hace falta un log particionado y cómputo en paralelo.
Almacenamiento	≈ 60 GB/día de JSON crudo, ≈ 10 GB/día en Delta comprimido ≈ 3,7 TB/año solo en Bronze. Con retención de 5 años son ≈ 18 TB más Silver y Gold.	Almacenamiento de objetos barato y separado del cómputo, con formato de tabla abierta.
Estado	Ventanas de 10 min y 1 h para 5,5 M clientes activos y un join con eventos de sesión en ±30 min: varios GB de estado vivo.	Estado particionado por `cliente_id` y distribuido entre executors, con checkpoint.
Variedad	Eventos JSON, maestros en Parquet, listas restrictivas en CSV y decisiones humanas de los analistas; los esquemas evolucionan.	Capa Bronze fiel a la fuente y schema enforcement al pasar a Silver.
Velocidad y disponibilidad	Alerta en ≤ 10 s, 24/7 (Bre-B no cierra).	Sin punto único de falla: replicación en Kafka y Cassandra.

Una base PostgreSQL «más grande» no resuelve el problema. Los escaneos analíticos de meses sobre varios TB compiten con el core transaccional, la escala es solo vertical, no se pueden reprocesar eventos del pasado y los triggers o el polling no calculan ventanas deslizantes con estado a 20.000 ev/s. El sistema necesita separar la **ruta caliente** (detectar en segundos) de la **ruta analítica** (lakehouse consultable), y cada una tiene requisitos de consistencia distintos (Sección 2).

Escala de la demo (E2): el simulador genera entre 200 y 2.000 ev/s en un portátil. Particiones, executors y nodos son parámetros, así que el diseño escala sin cambiar el código. El throughput por core se medirá y se extrapolará en E2.

2. Requisitos funcionales y no funcionales

2.1 Requisitos funcionales

La columna «Componente» usa los nombres del diagrama (Sección 3) para trazar cada requisito. [P1]–[P4] son los jobs de Spark.

ID	Requisito	Componente	Pregunta
RF-01	El sistema debe ingestar cada transacción confirmada por el core en el topic `tx.transacciones.v1`, con latencia de publicación p95 < 1 s desde el commit (patrón outbox).	Core → Kafka	Q1
RF-02	El sistema debe ingestar los eventos de sesión y dispositivo (login, cambio de dispositivo o clave, registro de llave Bre-B) en `app.eventos.v1`, particionados por `cliente_id`.	App → Kafka	Q1, Q3
RF-03	El sistema debe detectar la llegada de los archivos diarios de maestros, etiquetas y listas en `s3://landing/` y cargar cada archivo una sola vez a Bronze, validando el hash SHA-256 de su manifiesto.	MinIO landing, `dag_maestros_diario`	Q4, Q7
RF-04	El sistema debe persistir en Bronze el 100 % de los mensajes de todos los topics, sin alterar el payload, agregando `_topic, _partition, _offset, _ingest_ts`, en ≤ 60 s.	[P1] → Bronze	todas
RF-05	El sistema debe calcular por cliente, en ventanas deslizantes de 10 min (con paso de 1 min) y de 1 h, el número de transacciones, el monto acumulado, las contrapartes distintas y los dispositivos distintos.	[P2]	Q2, Q3
RF-06	El sistema debe correlacionar cada transacción con los eventos de sesión del mismo cliente ocurridos en los 30 minutos anteriores o posteriores (join stream-stream).	[P2]	Q1
RF-07	El sistema debe evaluar las reglas R1–R6 (Anexo B), calcular un score entre 0 y 1 y publicar en `fraude.alertas.v1` toda alerta con score ≥ 0,5, usando `alerta_id = sha256(tx_id ‖ regla_id)`.	[P2] → Kafka	Q1, Q3
RF-08	El sistema debe registrar cada alerta en Cassandra (`alertas_por_cliente`, `cola_alertas`) y servirla a la consola con p99 < 50 ms.	[P2] → Cassandra → Consola	Q1, Q2
RF-09	El sistema debe retener preventivamente, mediante el motor de bloqueo, las cuentas con una alerta ALTA (score ≥ 0,8), y registrar la acción con el `alerta_id`.	Motor de bloqueo	Q1
RF-10	El sistema debe capturar la decisión del analista (fraude, falso positivo o en investigación) en `fraude.decisiones.v1` y llevarla a `fact_alerta`.	Consola → Kafka → Gold	Q4, Q6
RF-11	El sistema debe validar tipos, montos > 0, moneda COP y existencia del cliente al pasar a Silver; debe enviar los registros inválidos a `silver.cuarentena` con el motivo del rechazo y deduplicar por `tx_id`.	[P3]	todas
RF-12	El sistema debe tokenizar el documento de identidad y el celular con HMAC-SHA-256 al pasar a Silver, de modo que Gold no contenga PII en claro.	[P3]	—
RF-13	El sistema debe mantener `dim_cliente` y `dim_regla` como SCD tipo 2 y actualizar `fact_transaccion` y `fact_alerta` con MERGE cada 15 min.	[P4] → Gold	Q4–Q8
RF-14	El sistema debe permitir consultar Gold con SQL en Trino y publicar tableros en Superset que respondan Q4, Q5, Q6 y Q8.	Trino, Superset	Q4–Q8
RF-15	El sistema debe generar todos los días a las 06:00 el reporte de candidatos a ROS para el Oficial de Cumplimiento.	`dag_cumplimiento_diario`	Q7
RF-16	El sistema debe recalcular a diario `perfil_riesgo` por cliente (promedio de 90 días, P95 de monto, municipio habitual, bandera de lista restrictiva) y cargarlo a Cassandra.	[P4] → Cassandra	Q2
RF-17	El sistema debe orquestar en Airflow todos los pasos batch y supervisar los jobs streaming. Ante un fallo, debe reintentar 3 veces y dejar las tareas siguientes en `upstream_failed`.	Airflow (4 DAGs)	—

2.2 Requisitos no funcionales

Escalabilidad

ID	Requisito	Mecanismo y verificación
RNF-01	La ruta caliente debe soportar **20.000 ev/s sostenidos durante 15 min** (pico de quincena) con latencia de alerta p95 ≤ 10 s, y recuperar el lag acumulado en ≤ 5 min después del pico.	12 particiones por topic, de modo que [P2] tiene hasta 12 tasks en paralelo. `maxOffsetsPerTrigger` limita la ingesta (backpressure). Se verifica con una prueba de carga del simulador.
RNF-02	El sistema debe escalar horizontalmente (añadiendo brokers, particiones, executors o nodos de Cassandra) sin cambiar el código. Ninguna partición de Cassandra debe superar 100 MB.	Sharding por `cliente_id` (hash del token ring). `cola_alertas` se reparte en 16 buckets para evitar una partición caliente.

Disponibilidad y posición CAP

La ruta caliente debe tener una disponibilidad del **99,9 % mensual** (≤ 43 min de caída al mes). Cada componente toma una posición CAP distinta según lo que cueste equivocarse en él:

Componente	CAP	Comportamiento ante partición de red o caída de nodos y por qué es lo correcto
Kafka	**CP**	Con `acks=all`, `min.insync.replicas=2` y `unclean.leader.election.enable=false`, si quedan menos de 2 réplicas sincronizadas el broker **rechaza escrituras** en vez de aceptar datos que podría perder. Es el registro de la verdad de los eventos, y el outbox del core retiene lo pendiente mientras tanto.
Delta Lake / MinIO	**CP**	Cada commit al `_delta_log` es atómico, con control de concurrencia optimista (aislamiento serializable de escritura). Si MinIO no está disponible, el job falla y se reintenta desde su checkpoint. Nunca queda una tabla a medio escribir.
Cassandra	**AP**	Con RF = 3 sigue aceptando lecturas y escrituras con un nodo caído. Escribe con `LOCAL_QUORUM` y lee con `LOCAL_ONE`; como R + W = RF, la lectura puede devolver un dato con segundos de atraso (PACELC: PA/EL). Para la consola, una cola con 1 s de atraso es aceptable; una consola caída en medio de un ataque no lo es.

Consistencia: garantía de entrega por tramo

Tramo	Garantía	Mecanismo
Core / app → Kafka	at-least-once	El productor idempotente (`enable.idempotence=true`) evita duplicados por reintentos. Si el relay del outbox se reinicia puede reenviar mensajes: los consumidores deduplican por `tx_id`.
Kafka → Bronze [P1]	**exactly-once**	Los offsets quedan en el checkpoint de Spark y el sink Delta es idempotente por `batchId` (el commit de datos y el de offsets son atómicos).
Kafka → alertas [P2]	at-least-once, **efecto exactly-once**	El sink Kafka de Spark no es transaccional: al reintentar un micro-batch se reenvían alertas. Como `alerta_id` es determinístico, el motor de bloqueo y Cassandra (upsert por clave primaria) son idempotentes.
Bronze → Silver → Gold	**exactly-once**	`Trigger.availableNow` con checkpoint y `MERGE` por llave natural (`tx_id`, `alerta_id`). Un reintento de Airflow no duplica filas.
Archivos landing → Bronze	exactly-once por archivo	Tabla de control de archivos procesados (ruta y SHA-256). Una recarga reemplaza la partición con `replaceWhere`.

Latencia

Ruta	Objetivo	Presupuesto por etapa
Evento → alerta en Kafka, Cassandra y motor de bloqueo	**p95 ≤ 10 s**, máx. 30 s	Outbox → Kafka ≤ 1 s · espera del trigger de [P2] ≤ 5 s · proceso del micro-batch ≤ 3 s · publicación ≤ 1 s
Evento → Bronze	≤ 60 s	Trigger de 30 s más el commit (archivos más grandes y menos small files)
Evento → disponible en Gold	**p95 ≤ 30 min**	Bronze ≤ 1 min · espera del ciclo de 15 min · [P3] + checks + [P4] ≤ 10 min
Consulta a la consola (Cassandra)	p99 < 50 ms	Lectura de una sola partición por `cliente_id`
Consulta analítica en Trino (90 días)	p95 < 15 s	Pruning por partición `fecha` y Z-ORDER por `cliente_key`

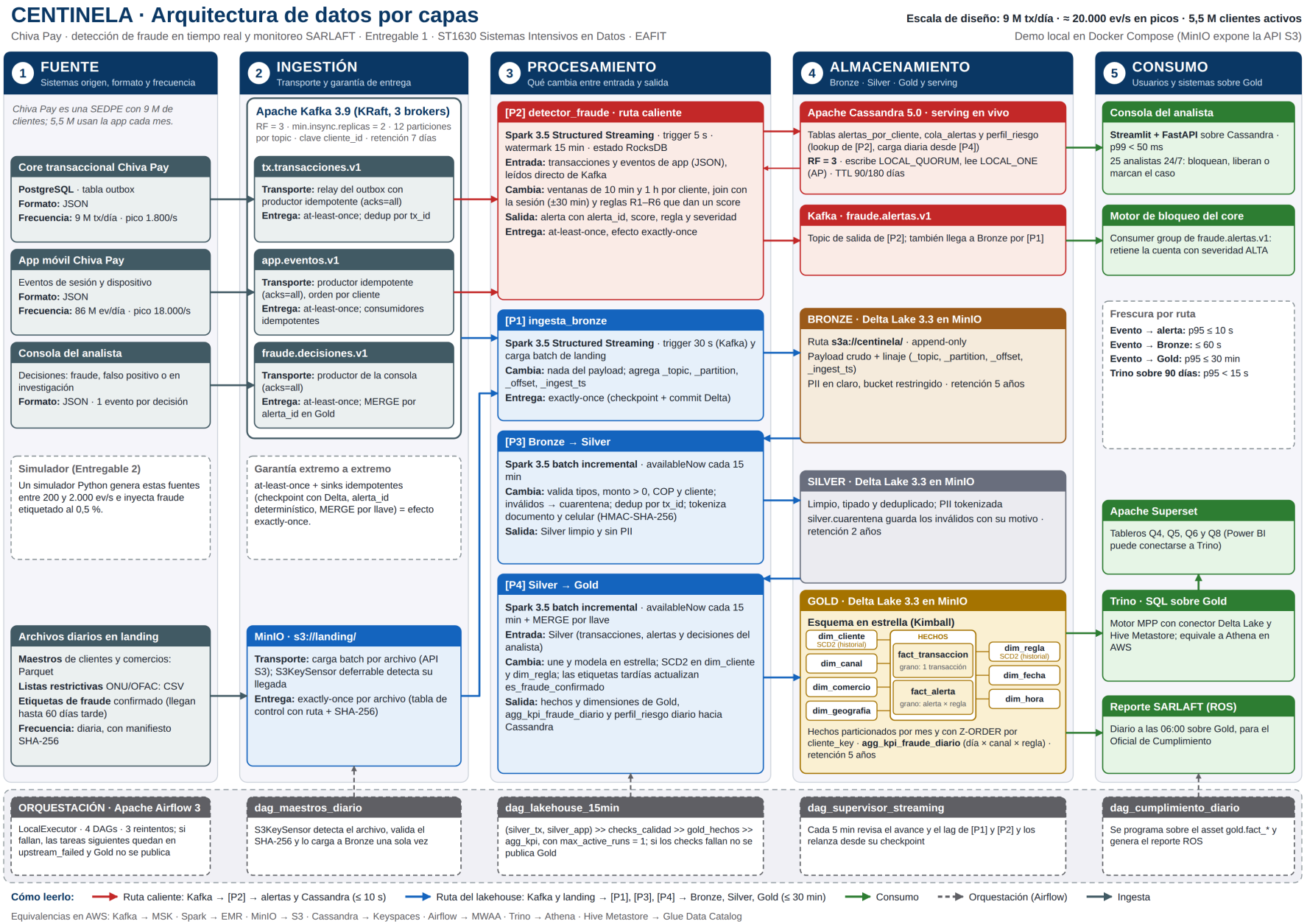
Tolerancia a fallos · RPO = 0 para todo evento confirmado por Kafka

Falla	Qué pasa y cómo se recupera	RTO
Cae 1 broker de Kafka	Un seguidor sincronizado pasa a líder. Con 2 réplicas en ISR se siguen aceptando escrituras y no se pierde nada.	< 30 s
Caen 2 brokers	Las escrituras se rechazan (CP). El outbox acumula en PostgreSQL y reenvía al volver el broker. Las alertas se retrasan, pero no se pierden.	al volver el broker
Falla el driver o un executor de [P2]	Docker reinicia el contenedor (`restart: unless-stopped`). Spark retoma desde el checkpoint: offsets y estado RocksDB. Si no vuelve, lo relanza `dag_supervisor_streaming`.	≤ 2 min / ≤ 5 min
Cae 1 nodo de Cassandra	Con RF = 3, `LOCAL_QUORUM` sigue operando con 2 nodos. El nodo que vuelve se pone al día con hinted handoff y repair.	0
Falla una tarea de Airflow	3 reintentos con backoff exponencial. Si se agotan, las tareas siguientes quedan en `upstream_failed`: Gold no se publica con datos incompletos y se envía una alerta al canal del equipo.	manual
Mensaje corrupto o esquema inválido	No detiene el pipeline: va a `silver.cuarentena` con el motivo. Bronze conserva el original para reprocesarlo.	—
Evento tardío (> 15 min)	Queda fuera de las ventanas de [P2] por el watermark, pero sí llega a Silver y Gold, y R1–R6 se reevalúan en el análisis diario.	—

Trazabilidad, seguridad y retención

- **Linaje de columnas:** cada fila lleva `_source_topic, _partition, _offset, _ingest_ts` (Bronze), `_bronze_version` (Silver) y `_run_id` de Airflow (Gold). A partir de una fila de `fact_transaccion` se puede llegar hasta el mensaje exacto de Kafka. `DESCRIBE HISTORY` y el time travel de Delta permiten reconstruir cómo estaba Gold en la fecha de un ROS.
- **Privacidad (Ley 1581 de 2012):** la PII en claro solo existe en Bronze, en un bucket con acceso restringido. Silver y Gold usan tokens HMAC. Los secretos se guardan en `.env`, fuera del repositorio.
- **Retención:** Kafka 7 días (ventana de replay) · Bronze y Gold 5 años (auditoría; supuesto alineado con SARLAFT) · Silver 2 años · Cassandra con TTL de 90 días (perfiles) y 180 días (alertas) · `VACUUM` de 168 h.

3. Diagrama de arquitectura de datos por capas



Archivos: `docs/entregable1/arquitectura.drawio`, `arquitectura.png`. **Cómo leerlo:** la **ruta roja** es la detección en tiempo real (Kafka → [P2] → alertas y Cassandra, ≤ 10 s). La **ruta azul** alimenta el lakehouse (Kafka y landing → [P1]/[P3]/[P4] → Bronze/Silver/Gold, ≤ 30 min). La **ruta verde** es el consumo. Airflow (banda inferior) dispara los pasos batch y supervisa los streaming.

4. Justificación de decisiones técnicas (ADRs)

Cada ADR sigue el formato de S10: contexto, alternativas, decisión y consecuencias. Todos quedan en estado *Aceptado* para el E2. Si alguno cambia durante la implementación, se registrará un ADR nuevo que lo reemplace.

ADR-01 · Motor de streaming: Apache Kafka con garantía at-least-once e idempotencia en los sinks

RF-01, RF-02, RF-07 · RNF-01

Contexto	Hay picos de 20.000 ev/s, al menos cuatro consumidores independientes del mismo evento ([P1], [P2], el motor de bloqueo y la consola), el requisito de reprocesar hasta 7 días y la necesidad de conservar el orden de los eventos de cada cliente, porque las ventanas y la regla de cambio de dispositivo dependen de la secuencia.
Alternativas	**Amazon Kinesis Data Streams:** es gestionado, pero cada shard admite ~1 MB/s o 1.000 registros/s de escritura (≥ 20 shards en pico), amarra el proyecto a AWS y no corre de forma nativa en local para la demo. **RabbitMQ:** tiene semántica de cola (el mensaje se borra al consumirse), así que no permite replay ni varios consumidores independientes sobre el mismo log sin duplicar colas. **Apache Pulsar:** es válido (separa cómputo y almacenamiento con BookKeeper), pero suma componentes, su integración con Spark es menos madura y no hace parte del curso.
Decisión	**Kafka 3.9 en modo KRaft, 3 brokers**, RF = 3, `min.insync.replicas=2`, sin elección de líder no limpia (CP). La clave de partición es `cliente_id`: garantiza el orden por cliente y ubica en la misma partición los eventos de tx y de app de un cliente, lo que reduce el shuffle del join. Productor con `acks=all` y `enable.idempotence=true`. La garantía extremo a extremo es **at-least-once** y el efecto exactly-once se consigue con sinks idempotentes: checkpoint + Delta y `alerta_id` determinístico. No se usan transacciones de Kafka (EOS) porque el sink Kafka de Spark no las soporta y los consumidores finales (Cassandra, core) están fuera de Kafka, así que igual necesitarían idempotencia. Los topics se versionan (`.v1`) y Spark lee con un esquema explícito.
Consecuencias	Hay que operar el clúster: 3 JVM, ~3 GB de RAM en la demo. Por ser CP, si caen 2 brokers se rechazan las escrituras y la latencia de las alertas sube (el outbox protege contra pérdida, pero no contra retraso). Todo consumidor debe tolerar duplicados. Sin Schema Registry, un cambio incompatible de esquema obliga a crear `.v2` y a correr ambos topics en paralelo (evaluar Schema Registry con Avro si hay tiempo). La regla R3 agrupa por receptor, lo que exige un shuffle que la clave no evita.

ADR-02 · Procesamiento: Spark Structured Streaming (micro-batch) en la ruta caliente y availableNow en el lakehouse

RF-05–RF-07, RF-11, RF-13 · latencia

Contexto	La alerta debe salir en ≤ 10 s p95 y requiere ventanas deslizantes con estado y un join stream-stream. Silver y Gold necesitan transformaciones batch pesadas (MERGE, SCD2, joins de 5 tablas). El proyecto lo desarrolla una sola persona, así que un solo motor y un solo lenguaje (PySpark) en todo el pipeline reducen a la mitad lo que hay que aprender y mantener.
Alternativas	**Apache Flink:** procesa evento por evento con latencias de milisegundos y tiene un estado muy robusto. Con un SLA de 10 s esos milisegundos no cambian la decisión del analista, y usarlo implica dos motores (Flink para streaming y Spark para batch), dos bases de código de reglas y PyFlink, que es menos maduro. **Kafka Streams:** es una librería JVM, no tiene API en Python y no sirve para el batch. **Batch puro cada 5 min:** incumple la latencia y la restricción de streaming del curso.
Decisión	**Spark 3.5 Structured Streaming.** [P2] lee *directamente de Kafka* (no desde Bronze) con un trigger de 5 s, un watermark de 15 min y un state store RocksDB. Esto evita sumar los 30 s de [P1] a la ruta caliente. [P1] es una consulta aparte, con un trigger de 30 s para escribir archivos más grandes en Bronze. [P3] y [P4] usan `Trigger.availableNow` lanzado por Airflow cada 15 min: procesan incrementalmente con checkpoint (semántica de streaming), pero liberan el clúster entre ejecuciones y permiten poner una compuerta de calidad entre Silver y Gold. Las reglas R1–R6 viven en un módulo Python compartido, de modo que el mismo código se reutiliza para reprocesar desde Bronze.
Consecuencias	La latencia mínima es el trigger más el tiempo de proceso (~5–8 s). Si el negocio pidiera menos de 1 s habría que migrar [P2] a Flink. Los eventos que llegan más de 15 min tarde se descartan de las ventanas. El estado crece con los clientes activos (RocksDB en disco y checkpoint en MinIO), y cambiar la lógica con estado obliga a versionar la ruta del checkpoint. Que dos consultas lean el mismo topic duplica la lectura en Kafka, un costo aceptable.

ADR-03 · Almacenamiento lakehouse: Delta Lake sobre MinIO y modelo en estrella en Gold

RF-04, RF-11–RF-14 · trazabilidad

Contexto	Se necesita ACID sobre almacenamiento de objetos: MERGE para deduplicar, aplicar SCD2 y reflejar etiquetas tardías de fraude; time travel para auditar reportes a la UIAF; schema enforcement y evolución. Gold lo consultan analistas con SQL, agrupando medidas por dimensiones.
Alternativas	**Apache Iceberg:** es excelente (particionamiento oculto, multi-motor). Se descarta por menor experiencia previa y porque los laboratorios del curso usan Delta; Trino soporta ambos, así que el cambio sería posible. **Apache Hudi:** está orientado a upserts, pero su configuración (CoW vs MoR, índices) añade complejidad sin beneficio para 15 min de frescura. **Parquet sin formato de tabla:** sin ACID ni MERGE, un lector puede ver escrituras a medias. Para el modelado de Gold: **One Big Table** (rápida, pero duplica atributos y pierde el historial de cliente y regla) y **Data Vault** (sobredimensionado para el consumo analítico de este caso).
Decisión	**Delta Lake 3.3** en `s3a://centinela/`, organizado en Bronze (append-only), Silver (limpio y deduplicado) y Gold. Gold es un **esquema en estrella** (Kimball) con dos hechos: `fact_transaccion` (grano: una transacción) y `fact_alerta` (grano: una alerta por regla). Comparten las dimensiones conformadas `dim_fecha, dim_hora, dim_cliente` (SCD2), `dim_canal, dim_comercio, dim_geografia` y `dim_regla` (SCD2) (detalle en el Anexo A). El modelo dimensional se ajusta porque Q4–Q8 son todas del tipo «medida por dimensión» (tasa de fraude por canal y hora, precisión por regla y semana). Además, SCD2 en `dim_regla` permite comparar la precisión *por versión de umbral*, algo que una OBT no permite. Los hechos se particionan por mes de `fecha` y se aplica Z-ORDER por `cliente_key`.
Consecuencias	MERGE reescribe archivos, por lo que hace falta un `OPTIMIZE` nocturno. Con concurrencia optimista, dos writers sobre la misma tabla chocan: Airflow corre con `max_active_runs=1`. Los hechos son mutables porque las etiquetas de fraude llegan hasta 60 días después y actualizan `es_fraude_confirmado`, así que una cifra histórica puede cambiar (y queda auditada con time travel). Se depende de Hive Metastore para que Trino vea las tablas.

ADR-04 · Componente libre: Apache Cassandra como capa de serving (AP)

RF-08, RF-16 · RNF-02 · CAP

Contexto	La consola y [P2] hacen búsquedas por cliente con p99 < 50 ms, con alta tasa de escritura (alertas en pico y 9 M perfiles recargados a diario), disponibilidad 24/7 y tolerancia a la caída de un nodo. Los patrones de acceso se conocen de antemano: (a) alertas de un cliente ordenadas por tiempo, (b) cola de alertas recientes, (c) perfil de riesgo por cliente. Delta y Trino tardan segundos por consulta, así que no sirven para esto.
Alternativas	**MongoDB:** un solo primario por replica set es CP por defecto; mientras se elige un nuevo primario (segundos) no acepta escrituras, y su flexibilidad de documentos e índices secundarios no aporta porque las consultas son fijas. **Redis:** latencia sub-ms, pero todo en memoria (9 M perfiles más 180 días de alertas es caro) y con durabilidad más débil; queda como caché futura, no como fuente. **DynamoDB:** modelo equivalente y gestionado, pero solo en AWS (ADR-06). **Neo4j:** ideal para investigar redes de mulas (trabajo futuro), pero no para lecturas clave-valor a esta tasa de escritura.
Decisión	**Cassandra 5.0**, RF = 3 (1 nodo en el perfil de demo, documentado), con diseño *query-first* y una tabla por patrón de acceso. **Partition key `cliente_id`** en `alertas_por_cliente` y `perfil_riesgo`: tiene alta cardinalidad (9 M), se distribuye de forma uniforme en el token ring y la lectura de la consola toca una sola partición. La clustering key es `ts_alerta DESC`. Si la partition key fuera `fecha`, todas las escrituras del día caerían en una sola partición caliente; si fuera `alerta_id`, el historial de un cliente exigiría scatter-gather. `cola_alertas` usa `((fecha, bucket))` con `bucket = hash(alerta_id) mod 16`, y la consola lee los 16 buckets en paralelo. Consistencia: escritura con `LOCAL_QUORUM` y lectura con `LOCAL_ONE`, lo que da una posición **AP** (PA/EL). La fuente de verdad de las decisiones sigue siendo Kafka y Delta, no Cassandra.
Consecuencias	No hay joins ni consultas ad hoc: los datos se desnormalizan y duplican por consulta, y un patrón de acceso nuevo implica una tabla nueva. La consistencia es eventual: con R + W = RF, un analista podría no ver durante ~1 s una alerta recién escrita. El TTL genera tombstones, por eso se usa TimeWindowCompactionStrategy en `cola_alertas`. Con 1 nodo en la demo no se puede mostrar la replicación; si la RAM alcanza, se levantan 3 nodos.

ADR-05 · Orquestación: Apache Airflow 3 con dependencias por tareas y por assets

RF-03, RF-15, RF-17

Contexto	Los pasos batch tienen dependencias (landing → Bronze → Silver → checks → Gold → reportes) y distintas cadencias (15 min y diaria). Los jobs streaming necesitan supervisión. La rúbrica exige que cada paso se dispare solo y que falle limpiamente si hay un error antes en la cadena.
Alternativas	**Dagster:** modelo por assets y buen linaje, pero es una herramienta nueva para mí y está fuera del curso. **Prefect:** simple, pero sus dependencias entre flujos son menos explícitas en la versión OSS. **cron + scripts:** no tiene grafo de dependencias, reintentos ni visibilidad. **Todo en streaming:** no resuelve los archivos diarios ni los reportes programados.
Decisión	**Airflow 3.x** (Airflow 2 llegó a fin de soporte en abril de 2026) con LocalExecutor y 4 DAGs. Dentro de cada DAG, las dependencias se definen con `>>`: `[silver_tx ‖ silver_app] >> checks_calidad >> gold_hechos >> agg_kpi`. Entre DAGs se usan **Assets**: `dag_lakehouse_15min` publica el asset `gold.fact_*` y `dag_cumplimiento_diario` se programa sobre él. Los archivos se detectan con un `S3KeySensor` en modo deferrable. Toda tarea es idempotente (availableNow con checkpoint o MERGE por llave), así que reintentar no duplica. La tarea de checks (conteos, nulos en FK, integridad referencial, frescura) funciona como compuerta: si falla, Gold no se publica. Los jobs streaming no son tareas porque corren indefinidamente: `dag_supervisor_streaming` revisa cada 5 min su progreso y el lag del consumer group, y los relanza desde el checkpoint.
Consecuencias	Airflow no es un orquestador de streaming: en el peor caso, recuperar [P2] tarda ~5 min. Por eso la primera línea es `restart: unless-stopped` de Docker y Airflow es la segunda. Con LocalExecutor, el scheduler es un punto único de falla en la demo (en producción se usarían 2 schedulers y CeleryExecutor o KubernetesExecutor). La base de metadatos es un componente más que operar.

ADR-06 · Plataforma: Docker Compose local con MinIO (compatible S3)

E2: demo en vivo, docker-compose

Contexto	El E2 exige una demo en vivo y un `docker-compose.yml` que levante el pipeline desde cero. El presupuesto es cero, hay un solo portátil de desarrollo y no se quiere depender de credenciales temporales ni de la red durante la sustentación.
Alternativas	**AWS gestionado** (MSK, EMR, S3, Keyspaces, MWAA): cercano a producción, pero MSK y MWAA cuestan cientos de USD al mes, AWS Academy impone límites y expira sesiones, y la demo quedaría expuesta a la red. **GCP** (Pub/Sub, Dataproc, BigQuery): mismas objeciones. **Databricks Community:** no permite Kafka propio ni Airflow.
Decisión	Todo corre en **Docker Compose**. MinIO expone la API S3, así que el código usa `s3a://` y migrar a AWS es cambiar el endpoint y las credenciales. Equivalencias: Kafka → MSK · Spark → EMR · MinIO → S3 · Cassandra → Amazon Keyspaces (CQL) · Airflow → MWAA · Trino → Athena · Hive Metastore → Glue Data Catalog.
Consecuencias	Se necesitan ~16 GB de RAM (Anexo C). Con todos los «nodos» en un mismo host, la replicación demuestra el comportamiento lógico ante fallas, pero no la independencia del hardware. El throughput se mide a escala reducida y se extrapola. Hay que operar todo localmente, incluidas las versiones de conectores.

ADR-07 · Consumo analítico: Trino + Apache Superset sobre Gold

RF-14 · Q4–Q8

Contexto	Riesgo y Cumplimiento hacen SQL ad hoc sobre Gold y necesitan tableros compartidos. Todo debe ser open source y correr en local (ADR-06).
Alternativas	**Spark Thrift Server:** reutiliza Spark, pero compite por recursos con [P2] (arriesga la latencia de la ruta caliente) y tiene arranque lento. **Athena:** está basado en Trino, pero es cloud. **Power BI:** requiere licencia y Windows (puede conectarse a Trino si el negocio lo exige). **DuckDB:** excelente en un solo nodo, pero no es un servicio multiusuario.
Decisión	**Trino** (motor MPP) con el conector Delta Lake sobre Hive Metastore, y **Superset** conectado a Trino. Separa el cómputo de consulta del cómputo de procesamiento: una consulta pesada de un analista no puede degradar la detección.
Consecuencias	Son ~2,5 GB de RAM y dos servicios más. Las tablas deben registrarse en HMS después de cada cambio de esquema. El acceso es solo de lectura para los analistas (Trino no escribe en Gold). No hay caché de resultados: los tableros usan `agg_kpi_fraude_diario` para no escanear los hechos.

5. Plan de implementación

5.1 Responsabilidades y alcance

El proyecto lo desarrolla un único integrante, **Sebastian Andres Medina Cabezas**, responsable de todos los componentes. Como no hay reparto de tareas, el riesgo principal es el alcance. Por eso el trabajo se divide en un **núcleo obligatorio**, que cubre todas las restricciones de la guía y es lo que se garantiza para el E2, y unas **extensiones** que solo se hacen si el núcleo ya funciona de extremo a extremo.

Componente	Núcleo obligatorio (garantizado en E2)	Extensión (si hay tiempo)
Fuentes e ingesta	Simulador Python con fraude etiquetado (ATO, mulas, fraccionamiento), Kafka con 3 brokers, productor idempotente, topics, landing en MinIO, [P1] `ingesta_bronze`	Patrón outbox real sobre PostgreSQL; Schema Registry
Streaming y detección	[P2] `detector_fraude`: ventanas, join stream-stream, reglas R1, R2, R3 y R5, sinks a Kafka y Cassandra, medición de latencia evento → alerta	Reglas R4 y R6; motor de bloqueo como consumer
Lakehouse	[P3] y [P4]: calidad, cuarentena, deduplicación, tokenización, SCD2 de `dim_cliente`, MERGE, estrella con `fact_transaccion` y `fact_alerta`	SCD2 de `dim_regla`; `agg_kpi_fraude_diario`
Serving (Cassandra)	Tablas `alertas_por_cliente`, `cola_alertas` y `perfil_riesgo` en 1 nodo; consulta CQL de la consola	3 nodos para mostrar la replicación; consola Streamlit + FastAPI
Orquestación	`dag_lakehouse_15min` con checks de calidad y `dag_maestros_diario`; fallo limpio en `upstream_failed`	`dag_supervisor_streaming`, `dag_cumplimiento_diario`
Consumo	Trino sobre Gold con consultas que responden Q4 y Q5	Superset; reporte ROS; Prometheus/Grafana

Si una extensión no se alcanza a implementar, se deja registrada en un ADR como decisión de alcance para la reflexión del E2. Ninguna extensión cambia la arquitectura del diagrama, solo el nivel de implementación.

5.2 Cronograma e hitos

Semana	Trabajo	Hito verificable
S12	Entrega del E1. Creación del repositorio. *Spike* de compatibilidad: Spark 3.5 + Delta 3.3 + conectores Kafka y Cassandra + Trino con imágenes de tag fijo.	**H0:** `versions.md` con la matriz validada
S13	Retroalimentación del E1. Compose del núcleo, simulador, [P1] a Bronze, esquema Cassandra.	**H1:** `docker compose up` levanta el núcleo y 500 ev/s llegan a Bronze
S14	[P2] con R1, R2, R3 y R5; [P3] con cuarentena y deduplicación. Asesoría con un problema técnico documentado.	**H2:** alertas en Kafka y Cassandra con p95 ≤ 10 s a 500 ev/s
S15	[P4] y la estrella; Trino; DAGs del núcleo con checks; pruebas de falla (tumbar un broker, matar [P2]). Extensiones solo si H3 y H4 están cumplidos.	**H3:** Q4 y Q5 responden en Trino · **H4:** un fallo inyectado deja las tareas siguientes en `upstream_failed`
S16 −1 día	Congelamiento de código, PR final, bitácora de IA, slides, ensayo cronometrado de 25 min con la demo completa.	**H5:** pipeline desde cero siguiendo solo el README

5.3 Riesgos técnicos

Riesgo	P / I	Mitigación
El alcance supera el tiempo disponible de una sola persona	Alta / Alto	Separación entre núcleo y extensiones (5.1); hitos semanales. Si en S14 H2 no está cumplido, se congelan todas las extensiones.
La RAM no alcanza para todos los servicios (~16 GB)	Alta / Alto	Perfil `demo` en Compose: Cassandra con 1 nodo, 1 worker Spark de 4 GB, Trino de 2 GB, sin Superset ni Grafana. Medirlo en S13. Plan B: una EC2 por horas.
Incompatibilidad de versiones entre Spark, Delta y los conectores	Media / Alto	Spike en S12, versiones fijadas en `versions.md`, prohibido usar `latest`.
No se cumple la latencia de ≤ 10 s	Media / Medio	Timestamps por etapa para ubicar el cuello de botella; ajustar `shuffle.partitions` a 12, el trigger y el tamaño del estado.
Las reglas no se disparan con datos sintéticos	Media / Medio	El simulador inyecta fraude a una tasa conocida (0,5 %) con ground truth para medir precisión y recall.
Checkpoint incompatible tras cambiar la lógica con estado	Media / Medio	Ruta de checkpoint versionada por job (`/checkpoints/p2/v3`) y procedimiento de reset documentado.
La demo falla en vivo	Media / Alto	Script de health-check, dataset precargado en modo replay desde Bronze, runbook de fallas y ensayo el día anterior.

Anexos (no cuentan en el límite de 15 páginas)

Anexo A · Modelo dimensional de Gold

Tabla	Grano	Columnas principales
fact_transaccion	1 fila = 1 transacción	tx_id (llave natural), fecha_key, hora_key, cliente_key, contraparte_key, canal_key, comercio_key, geografia_key · *medidas:* monto_cop, comision_cop · *flags:* tuvo_alerta, max_score, es_fraude_confirmado · _run_id
fact_alerta	1 fila = 1 alerta × regla	alerta_id, tx_id, fecha_key, cliente_key, regla_key (versión vigente al disparar) · *medidas:* score, monto_cop, seg_evento_a_alerta, seg_alerta_a_decision · decision, severidad
agg_kpi_fraude_diario	día × canal × regla	n_tx, monto_total, n_alertas, n_confirmadas, precision, monto_evitado, p95_latencia_alerta
dim_cliente (SCD2)	versión de cliente	cliente_key, cliente_id, doc_token, segmento, antigüedad, municipio_habitual, nivel_kyc, valid_from, valid_to, is_current
dim_regla (SCD2)	versión de regla	regla_key, regla_id (R1–R6), descripción, umbral, peso, valid_from, valid_to
dim_canal · dim_comercio · dim_geografia · dim_fecha · dim_hora	1 fila por miembro	canal (P2P, Bre-B, QR, recarga, corresponsal) · comercio_id, MCC, categoría · código DANE, municipio, departamento · calendario con quincena y festivo · franja horaria

Ejemplo para Q4: `SELECT r.regla_id, r.umbral, d.semana, count(*) alertas, avg(CASE WHEN a.decision='FRAUDE' THEN 1.0 ELSE 0 END) precision FROM fact_alerta a JOIN dim_regla r USING (regla_key) JOIN dim_fecha d USING (fecha_key) GROUP BY 1,2,3;`

Anexo B · Reglas de detección de la ruta caliente

ID	Patrón	Condición (parámetros iniciales, versionados en `dim_regla`)	Peso
R1	Velocidad	≥ 5 transacciones salientes en 10 min y monto acumulado > 3 × el promedio diario de 90 días	0,35
R2	Toma de cuenta (ATO)	Cambio de dispositivo o de clave en los 30 min previos + transferencia > $2.000.000 a una llave Bre-B nunca usada	0,50
R3	Cuenta mula	≥ 8 remitentes distintos en 1 h y reenvío de ≥ 80 % de lo recibido en < 30 min	0,45
R4	Contexto atípico	Transacción entre 00:00 y 05:00, desde un municipio distinto al habitual, por un monto > P95 del cliente	0,25
R5	Lista restrictiva	El origen o la contraparte tiene la bandera `en_lista` en `perfil_riesgo` (ONU/OFAC)	0,90
R6	Fraccionamiento	≥ 4 transacciones de montos similares (±5 %) en 1 h que suman > $10.000.000	0,30

score = $1 - \prod(1 - \text{peso}_i)$ sobre las reglas activadas. Severidad ALTA si el score es ≥ 0,8 y MEDIA si está entre 0,5 y 0,8.

Anexo C · Estimación de recursos (perfil demo)

Servicio	RAM
Kafka ×3 (KRaft combinado)	3,0 GB
Spark master + 1 worker (P1, P2 y batch)	5,0 GB
Cassandra ×1	2,0 GB
Airflow (API, scheduler, PostgreSQL)	2,0 GB
Trino + Hive Metastore	2,5 GB
MinIO, Superset, consola, Prometheus/Grafana	1,5 GB
Total	**≈ 16 GB**

Anexo D · Referencias

- Bloomberg Línea (2026). Fraude financiero en Colombia: panorama 2026 (cifra de Asobancaria, 1S-2025).
- Semana (2025). Fraude digital en Colombia se reduce a la mitad (TransUnion).
- LexisNexis Risk Solutions (2026). Verdadero Costo del Fraude, Colombia.
- Banco de la República. Bre-B, sistema de pagos inmediatos e indicadores.
- Apache Airflow. Upgrading to Airflow 3; Airflow 2 EOL: abril 2026.
- Documentación oficial de Kafka, Spark Structured Streaming, Delta Lake, Cassandra y Trino.
- Ley 1581 de 2012 (protección de datos personales); SFC, Circular Básica Jurídica, SARLAFT.